

CKM-Enabled Joint Transmission and Batching Optimization for Wireless LLM Services

Sixu Xiao*, Hongyang Du[†], Yong Zeng^{*†}

*National Mobile Communications Research Laboratory, Southeast University, Nanjing 210096, China

[†]Department of Electrical and Electronic Engineering, University of Hong Kong, Hong Kong SAR, China

[‡]Purple Mountain Laboratories, Nanjing 211111, China

230248845@seu.edu.cn, duhy@eee.hku.hk, yong_zeng@seu.edu.cn

Abstract—Wireless large language model (LLM) services exhibit a distinctive transmission-computation coupling. However, existing studies mainly treat batching in wireless LLM services as a static request-selection problem and typically rely on location-agnostic channel statistics, thereby overlooking both the temporal dynamics of queue evolution and the spatial heterogeneity of wireless channels. In this paper, we first develop a practical system model that jointly captures HARQ-based uplink transmission and cloud-side prefill batching. Based on this model, we formulate the joint transmission-and-batching control problem as a Markov decision process. To solve the resulting hybrid control problem, we develop a deep reinforcement learning algorithm that jointly optimizes transmit-power control and batch-size scheduling using CKM-augmented state information. Numerical results demonstrate that the proposed method improves effective throughput, mitigates transmission-induced tail latency, and enhances cloud-side batching efficiency.

Index Terms—Channel knowledge map, wireless LLM services, cross-layer optimization, deep reinforcement learning.

I. INTRODUCTION

Large language model (LLM) serving has been extensively studied from the perspective of batching and runtime scheduling. Representative advances include Orca [1], which introduces iteration-level scheduling and selective batching for transformer serving, and Sarathi-Serve [2], which mitigates the throughput–latency tradeoff via chunked prefills and stall-free scheduling. These studies have substantially improved the efficiency of LLM inference systems, but they mainly focus on the computation side and typically abstract away the impact of wireless access on request arrivals.

More recently, several works have started to jointly consider communication and computation in wireless LLM services [3], [4]. This line of work treats batching mainly as a request-selection and resource-allocation problem over an available request set, and focuses on throughput-oriented or per-request QoS-oriented objectives, rather than the long-term performance of a dynamic multi-request queueing system. The JPPO [5] further incorporates user-side transmit-power control, but the formulation remains essentially one-shot and static: transmission delay is modeled in a per-request manner without queue evolution or long-term system analysis.

Computing resources were provided by the Network and Information Center of Southeast University and Ascend.

Existing wireless LLM service studies have several limitations. **First**, they overlook the temporal dynamics of request arrivals. However, such dynamics are fundamental because batching gains arise only when multiple requests arrive within suitable time windows. Under realistic retransmission mechanisms, the uplink completion time is no longer a deterministic rate-based quantity, but a retransmission-induced random process. More specifically, transmission-side decisions affect not only the average uplink delay, but also the arrival-time distribution of requests to the cloud waiting queue, thereby reshaping batching opportunities and ultimately the tail time to first token (TTFT). Meanwhile, cloud-side batching decisions determine the urgency of serving newly arriving requests. **Second**, they use a set of location-agnostic channel statistics for the entire service region. Such a simplification overlooks the spatial heterogeneity of wireless channels across users and therefore fails to capture how location-dependent channel conditions shape the temporal arrival pattern of uploaded requests at the cloud waiting queue. In particular, requests from users under poor channel conditions tend to complete uplink transmission much later, and such requests are often responsible for tail latency, e.g., the P95/P99 TTFT.

Overall, existing works largely overlook the temporal dynamics of transmission-induced request arrivals and the spatial heterogeneity of user channels. These observations motivate a temporally coupled and location-aware framework that jointly optimizes transmission-side control and cloud-side batching. Adaptive transmission using instantaneous CSI improves reliability, but conventional channel estimation incurs extra overhead and latency, which is undesirable for latency-sensitive wireless LLM services. Channel knowledge maps (CKMs) [6] provide a location-indexed knowledge base that depicts the long-term structure of the channel and the environment. By providing such channel priors, the CKM offers a new spatial degree of freedom that allows the system to proactively shape uplink completion dynamics and, in turn, cloud-side batching opportunities.

The main contributions are summarized as follows:

- We develop a cross-layer system model for wireless LLM services that jointly captures HARQ-based uplink transmission and cloud-side prefill batching in a dynamic multi-request queueing system, and we formulate the

joint transmission-and-batching control problem.

- We recast the formulated problem as a discrete-time Markov decision process, and propose a CKM-enabled hybrid-action deep reinforcement learning algorithm to solve the problem. Source code is available at https://github.com/xxxxssxx/CKM_batching.
- We analyze the origin of the tail behavior of TTFT, reveal the sources of the performance gains achieved by the proposed method, and further demonstrate the necessity of both joint optimization and CKM support.

II. SYSTEM MODEL AND PROBLEM FORMULATION

A. System Model

We consider a cloud inference system over a service region, where users generate LLM requests over time and offload them to a cloud server through a base station (BS), as shown in Fig. 1. The lifecycle of each user request is as follows.

1) *Request Arrival*: User requests are generated over time within the service region according to a Poisson process with rate λ . Let t_i^{arr} denote the arrival time of request i , and let $q_i \in \mathcal{Q}$ denote the location of the user issuing request i , where $\mathcal{Q} \subset \mathbb{R}^2$ denotes the service region.

2) *Uplink Waiting*: Once generated, each request enters an uplink waiting queue Q_t^{ul} , where it waits for the BS to allocate wireless transmission resources. The queue is sorted in ascending order of t_i^{arr} .

3) *Uplink Transmission*: The BS can serve at most K users simultaneously on orthogonal uplink resources. Requests that are not scheduled remain in the uplink waiting queue. At time t , let $\mathcal{A}_t^{\text{ul}}$ denote the set of requests currently under uplink transmission, which satisfies $|\mathcal{A}_t^{\text{ul}}| \leq K$. The BS monitors $\mathcal{A}_t^{\text{ul}}$. If uplink resources are still available, i.e., $|\mathcal{A}_t^{\text{ul}}| < K$, the BS sequentially moves requests from the head of Q_t^{ul} into $\mathcal{A}_t^{\text{ul}}$. Each selected request is transmitted from the user to the BS via a HARQ-based uplink transmission process.

4) *Cloud Waiting*: After being successfully received by the BS, request i is forwarded to the cloud server and joins the cloud waiting queue Q_t^{cl} . Each request i in Q_t^{cl} is assigned a priority score $s_i(t)$, which may depend jointly on multiple factors such as request arrival time, urgency level, and communication-related status. Requests with larger scores are placed nearer the head of the queue and prioritized for inference. Therefore, cloud-side batch scheduling is not performed over an unordered request set, but over a dynamically reordered queue. For simplicity, we set

$$s_i(t) = -t_i^{\text{arr}}. \quad (1)$$

5) *Cloud Inference*: We consider a cloud server with N GPUs, each independently performing batching over queued requests. When a GPU becomes available, it fetches several requests from the head of Q_t^{cl} to form a batch and executes the prefill stage. For request i , let t_i^{fin} denote the completion time of the prefill stage of its batch.

For clarity, we adopt the following assumptions. (i) The BS-to-cloud and downlink transmission delays are negligible.

(ii) We focus only on the prefill stage, as it dominates TTFT and can be decoupled from decoding [7]. (iii) All requests have the same prompt length. (iv) Each GPU hosts a single model instance. Next, we detail the models of the main system components involved in the above request lifecycle.

1) *Location-aware Channel Model*: For a user located at position q , the channel power gain is modeled as a lognormal random variable

$$g(q) \sim \text{Lognormal}(a(q), b^2(q)), \quad (2)$$

or equivalently

$$\ln g(q) \sim \mathcal{N}(a(q), b^2(q)). \quad (3)$$

Let $\mu(q) = \mathbb{E}[g(q)]$ and $\sigma^2(q) = \text{Var}(g(q))$ denote the mean and variance of the channel gain in the linear domain. Then the corresponding lognormal parameters satisfy

$$a(q) = \ln \mu(q) - \frac{1}{2}b^2(q), \quad b^2(q) = \ln \left(1 + \frac{\sigma^2(q)}{\mu^2(q)} \right). \quad (4)$$

2) *CC-HARQ*: We adopt CC-HARQ in this work. Let $p_i^{(m)}$ denote the transmit power of request i in the m -th HARQ round, and let $g_i^{(m)}$ be the corresponding instantaneous channel gain. The received SNR in one transmission round is

$$\gamma_i^{(m)} = \frac{G_0 p_i^{(m)} g_i^{(m)}}{N_0 B_w}, \quad (5)$$

where N_0 is the noise power spectral density, B_w is the uplink bandwidth, and $G_0 > 0$ denotes a constant additional gain accounting for the array gain and beamforming gain. The accumulated SNR after m rounds is

$$\Gamma_i^{(m)} = \sum_{k=1}^m \gamma_i^{(k)} = \sum_{k=1}^m \frac{G_0 p_i^{(k)} g_i^{(k)}}{N_0 B_w}. \quad (6)$$

A request is successfully uploaded once

$$B_w \log_2(1 + \Gamma_i^{(m)}) \geq R, \quad (7)$$

where R is the fixed transmission rate. Let L_{ul} denote the coded block length in bits. Then each HARQ round occupies a fixed duration

$$\tau_{\text{ul}} = \frac{L_{\text{ul}}}{R}. \quad (8)$$

Within one HARQ round, we assume that the channel gain and the transmit power remain constant.

3) *Prefill Models*: We assume that all requests have the same prompt length L_p , measured in tokens. For a batch of size B , the prefill latency is modeled as: [2]

$$\tau_{\text{pr}}(B, L_p) = \beta_0 + \beta_1 B L_p + \beta_2 B L_p^2, \quad (9)$$

where β_0 captures the fixed launch overhead, the linear term accounts for MLP computation and KV-cache writing, and the quadratic term reflects the attention cost.

The GPU power consumption during prefill is modeled by a logistic function of the batch size B : [8]

$$P_{\text{pr}}(B) = P_{\min} + \frac{\Delta P}{1 + \exp(-k_1(\log_2 B - k_2))}, \quad (10)$$

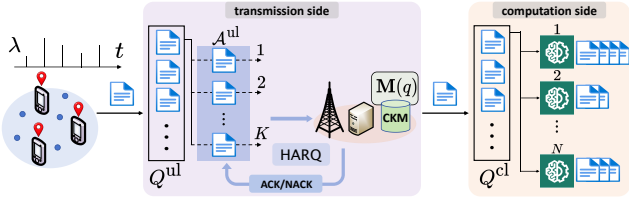


Fig. 1: System model of wireless LLM services.

for $B \geq 1$, while $P_{\text{pf}}(0) = 0$. Accordingly, the energy consumed by a launched batch is

$$E_{\text{pf}}(B, L_p) = P_{\text{pf}}(B) \tau_{\text{pf}}(B, L_p). \quad (11)$$

B. Performance Metrics

The primary user-centric metric is the TTFT, defined as

$$T_i^{\text{TTFT}} = t_i^{\text{fin}} - t_i^{\text{arr}}, \quad (12)$$

which consists of the uplink waiting delay, uplink transmission delay, cloud queueing delay, and prefill inference delay.

Let T denote the total system operating time, and let N_{srv} denote the number of requests served during T . The effective throughput is defined as

$$\eta_{\text{eff}} = \frac{1}{T} \sum_{i=1}^{N_{\text{srv}}} \mathbf{1}(T_i^{\text{TTFT}} \leq \tau_{\text{SLO}}), \quad (13)$$

where τ_{SLO} is the TTFT requirement specified by the service-level objective (SLO). This metric serves as a key proxy for revenue-generating throughput.

The average uplink energy per served request is

$$\bar{E}_{\text{ul}} = \frac{1}{N_{\text{srv}}} \sum_{i=1}^{N_{\text{srv}}} E_{\text{ul},i}, \quad E_{\text{ul},i} = \sum_{m=1}^{m_i} p_i^{(m)} \frac{L_{\text{ul}}}{R}, \quad (14)$$

where m_i is the number of HARQ rounds used by request i .

The average GPU energy per served request is

$$\bar{E}_{\text{pf}} = \frac{1}{N_{\text{srv}}} \sum_{m=1}^M E_{\text{pf},m}, \quad (15)$$

where M is the total number of launched batches and $E_{\text{pf},m}$ is the prefill energy of the m -th batch.

C. Problem Formulation

The system decisions at time t are the transmit power vector

$$\mathbf{p}(t) = [p_1(t), \dots, p_K(t)], \quad 0 \leq p_k(t) \leq p_{\text{max}}, \quad (16)$$

and the cloud-side batching vector

$$\mathbf{b}(t) = [B_1(t), \dots, B_N(t)], \quad B_n(t) \in \{0, 1, \dots, B_{\text{max}}\}. \quad (17)$$

Our goal is to find a causal control policy π that balances service profitability, latency-aware user experience, user uplink energy consumption, and cloud-side computation costs. Since η_{eff} measures the throughput of requests meeting the

TTFT requirement, thereby reflecting both service profitability and user experience, we formulate the problem as

$$\max_{\pi} \quad \eta_{\text{eff}} - w_1 \bar{E}_{\text{ul}} - w_2 \bar{E}_{\text{pf}}, \quad (18)$$

where $w_1, w_2 \geq 0$ are weighting coefficients.

III. THE PROPOSED METHOD

A. Problem Analysis

There are two key observations. (i) Uplink transmission reshapes the original request arrival sequence before cloud inference, so the buildup of Q^{ul} and Q^{cl} follows different temporal processes. Conversely, batching decisions determine the cloud-side service rate and backlog, thereby affecting the urgency of ongoing uplink transmissions. (ii) Batching is sensitive to tail behavior [9]. In the considered system, such tail behavior is induced by a few delayed uploads under poor channel conditions. Specifically, the uplink channel $g_i^{(m)}$ in Eq. (5) experienced by request i is governed by $\mu(q_i)$ and $\sigma^2(q_i)$. Consequently, requests from unfavorable locations may undergo poor uplink conditions and thus require much longer transmission completion times. Such delayed requests can postpone batch formation and increase the cloud-side waiting time of all requests in the same batch.

These observations lead to two implications. First, separately optimizing transmission and batching is insufficient, while static optimization cannot capture their closed-loop interaction over time. Second, the tail behavior of uplink transmission fundamentally arises from the spatial heterogeneity of wireless channels. The CKM is therefore beneficial, as it stores fine-grained channel knowledge, providing an additional degree of freedom for low-overhead location-aware transmission control to mitigate unfavorable uplink tails.

B. CKM-Enabled Location-Aware Control

To enable low-overhead location-aware control, we explicitly incorporate the location-specific channel information retrieved from the CKM into the system state. The CKM associated with the service region is represented as an $H \times W$ spatial grid aligned with the physical area, where each grid cell stores a J -dimensional vector that characterizes the channel statistics at the corresponding location. Denote the CKM by $\mathbf{M} \in \mathbb{R}^{H \times W \times J}$. For a location q in the service region, the corresponding CKM entry is denoted by $\mathbf{M}(q) \in \mathbb{R}^J$. In this work, we set $J = 2$, and $\mathbf{M}(q)$ contains the ground-truth parameters of the channel distribution at that location:

$$\mathbf{M}(q) = [\mu(q), \sigma^2(q)]. \quad (19)$$

When a user submits a request i , the user location q_i is also reported to the BS, and the BS queries the CKM to retrieve $\mu(q_i)$ and $\sigma^2(q_i)$. These location-dependent channel priors serve as part of the policy input.

C. MDP Formulation

We model the joint control problem as a discrete-time MDP for a continuous-time system. Specifically, the system evolves over a wall-clock horizon of length T , while decisions are made every Δt seconds. Let $\ell \in \{0, 1, \dots, L-1\}$ denote the decision-step index, where $L = \frac{T}{\Delta t}$, and let $t_\ell = \ell \Delta t$ be the corresponding real time. At each decision epoch t_ℓ , the controller observes the system state and takes an action. Therefore, both the state and the action are defined on the discrete decision steps, while the underlying system evolves continuously in wall-clock time.

1) *State Space*: The state space $\mathcal{S}$ characterizes the overall system status. The state at decision step ℓ is defined as

$$s_\ell = \left(|Q_{t_\ell}^{\text{ul}}|, |\mathcal{A}_{t_\ell}^{\text{ul}}|, |Q_{t_\ell}^{\text{cl}}|, \{s_{\ell,i}^{\text{ul}}\}_{i \in \mathcal{A}_{t_\ell}^{\text{ul}}}, s_\ell^{\text{q}}, \{\tau_{n,\text{rem}}^{\text{pf}}(t_\ell)\}_{n=1}^N \right). \quad (20)$$

For each request $i \in \mathcal{A}_{t_\ell}^{\text{ul}}$, we include

$$s_{\ell,i}^{\text{ul}} = \left(\mu_i, \sigma_i^2, m_i, \Gamma_i^{(m)}, d_i(t_\ell), \tau_{i,\text{rem}}^{\text{ul}}(t_\ell) \right), \quad (21)$$

where

$$d_i(t_\ell) = t_\ell - t_i^{\text{arr}} \quad (22)$$

is the waiting time since arrival, and $\tau_{i,\text{rem}}^{\text{ul}}(t_\ell)$ is the remaining uplink transmission time if the request is currently in service. Here, μ_i and σ_i^2 denote the channel statistics associated with request i . When the BS has access to the CKM, the parameters are the location-dependent quantities $\mu(q_i)$ and $\sigma^2(q_i)$ retrieved from the CKM. Otherwise, in the absence of a CKM, they reduce to the constants μ and σ^2 shared by the entire service region. Moreover, s_ℓ^{q} summarizes the waiting times of all requests currently stored in the two queues:

$$s_\ell^{\text{q}} = \left(\{d_i(t_\ell)\}_{i \in Q_{t_\ell}^{\text{ul}}}, \{d_i(t_\ell)\}_{i \in Q_{t_\ell}^{\text{cl}}} \right). \quad (23)$$

Here, $\tau_{n,\text{rem}}^{\text{pf}}(t_\ell)$ denotes the remaining execution time of the current prefill batch on GPU n . Therefore, the state captures both the transmission-side and computation-side system states, providing sufficient information for cross-layer control.

2) *Action Space*: The action space $\mathcal{A}$ consists of hybrid control actions over the transmission and computation processes. The action taken at decision step ℓ is defined as

$$a_\ell = (\mathbf{p}_\ell, \mathbf{b}_\ell) \quad (24)$$

where $\mathbf{p}_\ell$ and $\mathbf{b}_\ell$ correspond to the transmit power vector in (16) and the batching vector in (17), respectively.

In particular, the power-control action is applied only to requests that need to start a new HARQ round at real time t_ℓ , whereas the batching action is applied only to GPUs that are idle at real time t_ℓ .

3) *State Transition*: At decision epoch t_ℓ , the controller observes the current state s_ℓ and applies the action $a_\ell = \{\mathbf{p}_\ell, \mathbf{b}_\ell\}$, where $\mathbf{p}_\ell$ controls the transmit powers for newly initiated HARQ rounds and $\mathbf{b}_\ell$ determines the batch sizes for idle GPUs. During the interval $[t_\ell, t_{\ell+1})$, the system evolves according to the request lifecycle in Sec. II-A: new requests may arrive, ongoing uplink transmissions may complete one

HARQ round, and ongoing prefill batches may complete service. These events update the uplink waiting queue Q^{ul} , the active uplink set $\mathcal{A}^{\text{ul}}$, the cloud waiting queue Q^{cl} , and the GPU service states. At the next decision epoch $t_{\ell+1}$, the next state $s_{\ell+1}$ is constructed based on the evolved system.

Therefore, the state transition is compactly represented as

$$s_{\ell+1} = f(s_\ell, a_\ell, \xi_\ell), \quad (25)$$

where ξ_ℓ collects the exogenous randomness over $[t_\ell, t_{\ell+1})$, including request arrivals and channel realizations. As characterized by (25), the transition kernel $\mathcal{P}(s_{\ell+1} | s_\ell, a_\ell)$ depends only on the current state-action pair (s_ℓ, a_ℓ) . Hence, the controlled process is Markov.

4) *Reward*: According to Eq. (13), η_{eff} can be written as

$$\eta_{\text{eff}} = \frac{1}{T} \sum_{\ell=0}^{L-1} N_\ell^{\text{SLO}}, \quad (26)$$

where N_ℓ^{SLO} denotes the number of requests whose prefill is completed at decision step ℓ and whose TTFT satisfies the SLO constraint. Moreover, let $\eta_{\text{srv}} = \frac{N_{\text{srv}}}{T}$ denote the overall service throughput. Then, according to Eq. (14) and Eq. (15), the cumulative uplink and prefill energies satisfy

$$\sum_{\ell=0}^{L-1} E_\ell^{\text{ul}} = T \eta_{\text{srv}} \bar{E}_{\text{ul}}, \quad \sum_{\ell=0}^{L-1} E_\ell^{\text{pf}} = T \eta_{\text{srv}} \bar{E}_{\text{pf}}. \quad (27)$$

Hence, Eq. (18) is equivalent to

$$\max_{\pi} \sum_{\ell=0}^{L-1} N_\ell^{\text{SLO}} - \frac{w_1}{\eta_{\text{srv}}} \sum_{\ell=0}^{L-1} E_\ell^{\text{ul}} - \frac{w_2}{\eta_{\text{srv}}} \sum_{\ell=0}^{L-1} E_\ell^{\text{pf}}. \quad (28)$$

However, the factor η_{srv} becomes available only after the entire horizon is realized, and therefore cannot be directly used for online decision-making. Therefore, we adopt the following immediate reward r_ℓ :

$$r_\ell = \alpha N_\ell^{\text{SLO}} - \beta E_\ell^{\text{ul}} - \gamma E_\ell^{\text{pf}}, \quad (29)$$

where $\alpha, \beta, \gamma \geq 0$ are weighting factors. This reward is a tractable surrogate for the original long-term objective, obtained by removing the trajectory-dependent factor η_{srv} .

D. Hybrid-Action PPO

We employ PPO [10] for policy optimization due to its simplicity and training stability. The considered control problem involves a hybrid action space, consisting of continuous power control and discrete batch scheduling.

1) *Actor*: Since not all action dimensions are valid at every decision epoch, we further introduce action masks. Specifically, let $m_{\ell,i}^{\text{p}} \in \{0, 1\}$ indicate whether the i -th uplink slot needs to start a new HARQ round at step ℓ , and let $m_{\ell,n}^{\text{b}} \in \{0, 1\}$ indicate whether GPU n is idle and can launch a new batch. The overall policy is then written as

$$\pi_\theta(a_\ell | s_\ell) = \prod_{i=1}^K \left[\pi_{\theta,i}^{\text{p}}(p_{\ell,i} | s_\ell) \right]^{m_{\ell,i}^{\text{p}}} \prod_{n=1}^N \left[\pi_{\theta,n}^{\text{b}}(b_{\ell,n} | s_\ell) \right]^{m_{\ell,n}^{\text{b}}}. \quad (30)$$

For the continuous power-control branch, the actor outputs the mean of a Gaussian distribution for each uplink action dimension, while the log-standard deviation is learned as a global parameter independent of the state. Let $u_{\ell,i}$ denote the pre-squashed latent variable. Then

$$u_{\ell,i} \sim \mathcal{N}(\mu_{\theta,i}(s_\ell), \sigma_{\theta,i}^2), \quad i = 1, \dots, K. \quad (31)$$

To satisfy the peak-power constraint, the sampled latent action is squashed by a tanh function and then affine-transformed to the feasible interval $[0, p_{\max}]$, i.e.,

$$p_{\ell,i} = \frac{p_{\max}}{2} (1 + \tanh(u_{\ell,i})). \quad (32)$$

For the cloud-side batching branch, the actor outputs categorical logits for each GPU over a finite set of candidate batch sizes. Let $\mathcal{B} = \{0, 1, \dots, B_{\max}\}$ denote the candidate set. The batch policy for GPU n is given by

$$\pi_{\theta,n}^b(b_{\ell,n}|s_\ell) = \text{Categorical}(\mathbf{z}_{\theta,n}(s_\ell)), \quad b_{\ell,n} \in \mathcal{B}. \quad (33)$$

$$\mathbf{z}_{\theta,n}(s_\ell) = [z_{\theta,n,1}(s_\ell), \dots, z_{\theta,n,|\mathcal{B}|}(s_\ell)] \in \mathbb{R}^{|\mathcal{B}|},$$

where $\mathbf{z}_{\theta,n}(s_\ell)$ denotes the logit vector output by the actor for GPU n over the candidate batch-size set $\mathcal{B}$.

2) *Critic*: The critic estimates the state value

$$V_\phi(s_\ell), \quad (34)$$

which is used for generalized advantage estimation (GAE).

3) *Update*: Given a rollout trajectory, the temporal-difference residual is

$$\delta_\ell = r_\ell + \gamma_{\text{rl}} V_\phi(s_{\ell+1}) - V_\phi(s_\ell), \quad (35)$$

where $\gamma_{\text{rl}} \in (0, 1)$ is the reward discount factor. The GAE advantage is computed backward as

$$\hat{A}_\ell = \delta_\ell + \gamma_{\text{rl}} \lambda_{\text{gae}} \hat{A}_{\ell+1}, \quad (36)$$

where $\lambda_{\text{gae}} \in [0, 1]$ is the GAE parameter.

Let ϕ_{old} and θ_{old} denote the parameters used for rollout. The regression target for the critic is then

$$\hat{R}_\ell = \hat{A}_\ell + V_{\phi_{\text{old}}}(s_\ell). \quad (37)$$

The PPO probability ratio is defined as

$$\rho_\ell(\theta) = \frac{\pi_\theta(a_\ell|s_\ell)}{\pi_{\theta_{\text{old}}}(a_\ell|s_\ell)}. \quad (38)$$

The clipped surrogate objective is

$$\mathcal{L}_{\text{clip}}(\theta) = \mathbb{E}_\ell \left[\min \left(\rho_\ell(\theta) \hat{A}_\ell, \text{clip}(\rho_\ell(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_\ell \right) \right], \quad (39)$$

where $\epsilon > 0$ is the clipping coefficient. The actor is updated by minimizing

$$\mathcal{L}_{\text{actor}}(\theta) = -\mathcal{L}_{\text{clip}}(\theta) - c_e \mathbb{E}_\ell[\mathcal{H}_\ell], \quad (40)$$

where $\mathcal{H}_\ell$ is the policy entropy and $c_e \geq 0$ is the entropy coefficient. The critic is trained by minimizing the mean-squared error

$$\mathcal{L}_{\text{critic}}(\phi) = c_v \mathbb{E}_\ell \left[\left(V_\phi(s_\ell) - \hat{R}_\ell \right)^2 \right], \quad (41)$$

where $c_v \geq 0$ is the critic-loss coefficient.

IV. EXPERIMENTS AND NUMERICAL RESULTS

A. Simulation Setup

Simulations are conducted over a selected service region from CKMImageNet [11], as shown in Fig. 2. The reward parameters in Eq. (29) are set to $\alpha = 2$, $\beta = 20$, and $\gamma = 0.1$ to balance the differences in magnitude among the three terms. $p_{\max}$ is set to 0.4 W, and requests are generated only at locations satisfying $\mu_{\text{dB}} > -150$ dB, to bound the number of transmission rounds by 50. τ_{SLO} is set to 0.15 s, $B_{\max}$ is set to 32. All other system parameters and neural network architectures are provided in the codebase.

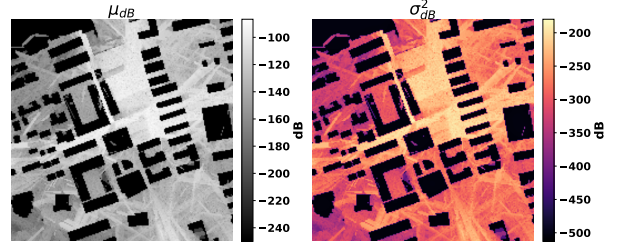


Fig. 2: Selected service region and the corresponding CKM.

B. Spatially Induced Tail Behavior of Uplink Transmission

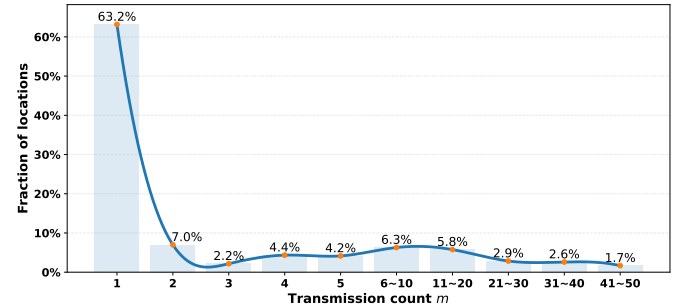


Fig. 3: Distribution of the required HARQ rounds for successful uplink delivery under fixed transmit power $p_{\max}$.

Fig. 3 shows the distribution of the HARQ round count across the selected area when the transmit power is fixed at $p_{\max}$. While most requests can be delivered within a small number of HARQ rounds, a non-negligible fraction requires substantially more retransmissions. This confirms that spatially varying channel conditions induce a long tail in uplink completion time. As discussed in Sec. III-A, this uplink-side tail is a fundamental source of the tail TTFT behavior in the overall system.

C. Comparative Study

Existing wireless LLM batching methods are not directly comparable because they mainly rely on static request-selection formulations without dynamic queue evolution or HARQ-induced uplink completion dynamics. Hence, we construct two ablation baselines to isolate the effects of joint optimization and CKM support. **Baseline 1**: the transmission

TABLE I: Performance comparison of different methods.

Method	$\frac{\eta_{\text{eff}}}{\lambda}$	$\bar{E}_{\text{ul}}$	$\bar{E}_{\text{pf}}$	$\bar{B}$	P99 TTFT	$\bar{T}_{\text{tx}}^{(1\%)}$	$\bar{T}_{\text{cp}}^{(1\%)}$
Proposed	99.2%	1.272×10^{-3}	0.5333	15.7	0.1352	0.1167	0.0370
Baseline 1	89.5%	1.239×10^{-3}	0.5419	9.4	0.1699	0.1448	0.0427
Baseline 2	82.0%	1.318×10^{-3}	0.5386	11.1	0.2376	0.2190	0.0386

side and the computation side are optimized separately with CKM support. **Baseline 2:** the transmission side and the computation side are jointly optimized without CKM support.

Let $\eta_{\text{eff}}/\lambda$ denote the effective service ratio, i.e., the fraction of arriving requests that are successfully served within the SLO constraint, and $\bar{B}$ denote the average batch size over all launched batches. We define the transmission-side latency as the sum of the durations in Sections II-A2 and II-A3, and the computation-side latency as the sum of the durations in Sections II-A4 and II-A5. Accordingly, $\bar{T}_{\text{tx}}^{(1\%)}$ and $\bar{T}_{\text{cp}}^{(1\%)}$ denote the average transmission-side and computation-side latency, respectively, over the 1% of requests with the largest TTFT values. Table I summarizes the results.

Overall, the proposed method achieves the best performance in almost all metrics. In particular, the proposed method achieves the highest $\eta_{\text{eff}}/\lambda$ and a P99 TTFT below τ_{SLO} , demonstrating strong tail-latency performance. Meanwhile, the proposed method attains a larger $\bar{B}$ and a lower $\bar{E}_{\text{pf}}$, indicating higher resource efficiency.

Compared with Baseline 1, the proposed method improves all metrics except for a slight increase in $\bar{E}_{\text{ul}}$. Under Baseline 1, the transmission side can leverage CKM to reduce uplink energy consumption, but the transmission policy is determined without considering the computation-side state. As a result, the uplink control cannot actively shape the request arrival pattern at the cloud waiting queue to create favorable batching opportunities, thereby degrading the computation-side performance. The smaller $\bar{B}$ indicates that the cloud side has to launch prefill more frequently to avoid queue buildup, which in turn increases $\bar{E}_{\text{pf}}$. By contrast, the proposed method uses slightly more uplink energy as a tradeoff to improve the overall system performance.

Compared with Baseline 2, the proposed method achieves better performance across all metrics. In particular, Baseline 2 incurs a much larger $\bar{T}_{\text{tx}}^{(1\%)}$, indicating that the tail bottleneck mainly lies on the transmission side. Without CKM, the initial transmission policy, particularly the transmit power for the first HARQ round, is identical for all requests. Such a location-agnostic design wastes transmit power for requests under favorable channel conditions, while causing requests under unfavorable channel conditions to undergo more retransmissions. Consequently, Baseline 2 suffers from both higher uplink energy consumption and much longer tail transmission latency.

In summary, joint optimization mitigates the performance loss caused by uncoordinated cross-layer decisions, while CKM mitigates the uplink tail caused by spatial channel heterogeneity. Moreover, the benefit of CKM is not limited to the

transmission side: CKM-enabled location-aware transmission control improves cloud-side batching efficiency by reshaping the request arrival process at the cloud waiting queue.

V. CONCLUSION

This paper investigated CKM-enabled cross-layer control for wireless LLM services. Numerical results show that, in temporally coupled systems, spatial channel heterogeneity is a fundamental source of uplink tail latency, and the resulting delayed uploads further degrade cloud-side batching opportunities and overall system performance. The results also verify the importance of jointly optimizing the transmission side and the computation side with location-aware channel knowledge for latency-sensitive wireless LLM services.

REFERENCES

- [1] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for Transformer-based generative models," in *Symp. Operating Syst. Des. Implementation (OSDI 22)*, Carlsbad, CA, Jul. 2022, pp. 521–538.
- [2] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. Gulavani, A. Tumanov, and R. Ramjee, "Taming throughput-latency tradeoff in LLM inference with sarathi-serve," in *Symp. Operating Syst. Des. Implementation (OSDI 24)*, 2024, pp. 117–134.
- [3] X. Zhang, J. Liu, Z. Xiong, Y. Huang, G. Xie, and R. Zhang, "Edge intelligence optimization for large language model inference with batching and quantization," in *Proc. Wireless Commun. Netw. Conf. (WCNC)*, 2024, pp. 1–6.
- [4] X. Zhang, J. Nie, Y. Huang, G. Xie, Z. Xiong, J. Liu, D. Niyato, and X. Shen, "Beyond the cloud: Edge inference for generative large language models in wireless networks," *IEEE Trans. Wireless Commun.*, vol. 24, no. 1, pp. 643–658, 2025.
- [5] F. You, H. Du, K. Huang, and A. Jamalipour, "JPPO: Joint power and prompt optimization for accelerated large language model services," in *Proc. IEEE Int. Conf. Commun.*, 2025, pp. 1578–1583.
- [6] Y. Zeng, J. Chen, J. Xu, D. Wu, X. Xu, S. Jin, X. Gao, D. Gesbert, S. Cui, and R. Zhang, "A tutorial on environment-aware communications via channel knowledge map for 6G," *IEEE Commun. Surv. Tut.*, vol. 26, no. 3, pp. 1478–1519, 2024.
- [7] E. Chouksey, P. Patel, C. Zhang, A. Shah, I. Goiri, S. Maleki, R. Fonseca, and R. Bianchini, "Splitwise: Efficient generative LLM inference using phase splitting," *IEEE Micro*, vol. 45, no. 4, pp. 54–59, 2025.
- [8] Z. Liang, J.-W. Chung, M. Chowdhury, J. Chen, and V. Dvorkin, "GPU-to-Grid: Voltage regulation via GPU utilization control," 2026. [Online]. Available: <https://arxiv.org/abs/2602.05116>
- [9] Y. Yang, L. Jiao, and Y. Xu, "A queueing theoretic perspective on low-latency LLM inference with variable token length," in *Int. Symp. Model. Optim. Mob. Ad Hoc Wireless Net. (WiOpt)*, 2024, pp. 273–280.
- [10] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017. [Online]. Available: <https://arxiv.org/abs/1707.06347>
- [11] Z. Wu, D. Wu, S. Fu, Y. Qiu, and Y. Zeng, "CKMImageNet: A dataset for AI-based channel knowledge map toward environment-aware communication and sensing," *IEEE Trans. Commun.*, vol. 73, no. 12, pp. 14 430–14 443, 2025.